LegalEase - Project Layout

Phase 0 — Project Scaffold

Prompt 1 — Project Setup

> "Set up a Flask + MongoDB project called LegalEase. Create the full folder structure (`app/`, `templates/`, `static/`, `uploads/`), `config.py` with environment-based settings (`SECRET_KEY`, `MONGO_URI`, `UPLOAD_FOLDER`), `__init__.py` with the Flask app factory pattern using PyMongo, and a `requirements.txt` including flask, pymongo, python-dotenv, flask-login, Pillow, pytesseract, and pdfplumber. Include a working `run.py` entry point."

🧪 Test — Prompt 1

> "Write a test script `test_setup.py` that: (1) imports the Flask app and asserts it is not None, (2) checks that all required config keys (`SECRET_KEY`, `MONGO_URI`, `UPLOAD_FOLDER`) are present and non-empty, (3) performs a live MongoDB ping using `pymongo` and asserts the connection succeeds, (4) checks that the `uploads/` directory exists and is writable, (5) verifies all packages in `requirements.txt` are importable. Run it and show the output."

Phase 1 — Authentication & User Management

Prompt 2 — Register, Login & Sessions

> "In LegalEase (Flask + MongoDB), implement user registration and login. Create a `users` MongoDB collection with fields: `_id`, `username`, `email`, `password_hash`, `role` (default: 'user'), `created_at`. Use Flask-Login for session management. Build `/register` and `/login` routes with form validation, bcrypt password hashing, and redirect to a `/dashboard` page on success. Show flash messages for errors."

🧪 Test — Prompt 2

> "Write tests for the auth system using Flask's test client. Cover these cases: (1) successful registration with valid data returns a 302 redirect to `/dashboard`, (2) registering with a duplicate email returns the registration page with an error flash message, (3) registering with a missing field returns a 400 with the appropriate error, (4) login with correct credentials creates a valid session and redirects to `/dashboard`, (5) login with wrong password returns an error flash, (6) accessing `/dashboard` without logging in redirects to `/login`, (7) logging out clears the session and redirects to `/login`. Use an isolated test MongoDB database and clean it up after tests."

Prompt 3 — Profile Management Page

> "Add a `/profile` route in LegalEase where a logged-in user can view and update their name, email, and password. Build the profile page template with editable fields, client-side validation, and a backend PATCH handler that updates the MongoDB user document. Show a success confirmation message on save."

🧪 Test — Prompt 3

> "Write tests for the profile page: (1) visiting `/profile` while logged out redirects to `/login`, (2) the profile page loads and pre-fills the form with the current user's name and email, (3) a valid update request changes the user's name in MongoDB and shows a success flash, (4) submitting an email already used by another user returns an error without modifying the record, (5) submitting a new password that doesn't meet length requirements returns a validation error, (6) updating the password stores a new bcrypt hash (not plaintext) in MongoDB. Confirm the old password hash is invalidated after a password change."

Phase 2 — Document Upload & Processing Pipeline

Prompt 4 — Document Upload UI & Backend Handler

> "In LegalEase, implement document uploading at `/upload`. Build an HTML upload form accepting PDF, PNG, and JPG only (client and server-side validation). The Flask backend should save the file to the `uploads/` folder with a UUID filename, create a `documents` MongoDB collection entry with fields: `doc_id`, `user_id`, `original_filename`, `stored_filename`, `file_type`, `upload_date`, `status` (default: 'uploaded'), and return JSON with the `doc_id`."

🧪 Test — Prompt 4

> "Write upload tests: (1) uploading a valid PDF returns a JSON response with a `doc_id` and the file is present in `uploads/`, (2) uploading a valid PNG works the same way, (3) uploading a `.exe` or `.txt` file returns a 400 error and no MongoDB record is created, (4) uploading a 0-byte file returns an error, (5) uploading without being logged in returns a 401 or redirect, (6) the MongoDB `documents` record has `user_id` correctly set to the logged-in user's `_id`, (7) uploading two files with the same original name results in two different UUID-named files on disk with no collision."

Prompt 5 — OCR for Image-Based Uploads

> "In LegalEase, add an OCR processing step for PNG and JPG uploads. After a file is saved, if it's an image, run it through pytesseract to extract raw text. Handle edge cases: low-resolution images (add a DPI check), skewed text (use pytesseract's OSD for orientation detection). Store the extracted raw text in the document's MongoDB entry under `raw_text`. Update the document `status` to 'ocr_complete'."

🧪 Test — Prompt 5

> "Write OCR tests: (1) upload a clear PNG of a printed legal paragraph and assert `raw_text` in MongoDB is non-empty and contains expected keywords, (2) upload a low-resolution image (under 150 DPI) and assert the system either upscales it or returns a warning flag in the document record, (3) upload a skewed image and assert OSD is invoked and corrects orientation before OCR, (4) upload a blank white image and assert `raw_text` is an empty string and `status` is set to 'ocr_failed' or a suitable fallback status, (5) assert that a PDF upload does NOT trigger the OCR path by checking no pytesseract call is made (use mocking)."

Prompt 6 — PDF Text Extraction & Text Cleaning Pipeline

> "In LegalEase, add PDF text extraction using pdfplumber for PDF uploads. Then build a shared text-cleaning pipeline (works for both OCR output and PDF text) that strips headers, footers, and page

> numbers using regex patterns, normalizes whitespace, and removes boilerplate. After cleaning, build a rule-based document type classifier that detects: Contract, NDA, Terms & Conditions, Privacy Policy, or Unknown — based on keyword frequency. Store `cleaned_text` and `doc_type` in MongoDB and update status to 'processed'."

🧪 Test — Prompt 6

> "Write pipeline tests: (1) run the PDF extractor on a sample multi-page PDF and assert `raw_text` contains content from multiple pages, (2) run the cleaner on text with known headers/footers (e.g. 'Page 1 of 10') and assert they are absent from `cleaned_text`, (3) run the document classifier on a text containing 'non-disclosure', 'confidential', 'disclosing party' and assert `doc_type` = 'NDA', (4) run it on a text with 'terms of service', 'user agreement', 'by using this website' and assert 'Terms & Conditions', (5) run it on a blank or irrelevant text and assert 'Unknown', (6) assert the entire pipeline triggers end-to-end on a fresh PDF upload and the final MongoDB status is 'processed'."

Phase 3 — Document History & Management

Prompt 7 — Document History Page

> "In LegalEase, build a `/documents` route that shows a logged-in user's document history. Fetch all documents from MongoDB linked to `current_user.id`, display them in a table with filename, upload date, doc type, and status. Add client-side filtering by doc type and sorting by date. Include action buttons per row: View, Re-analyze, and Delete (with a confirmation modal). Implement the delete endpoint and re-analyze endpoint."

🧪 Test — Prompt 7

> "Write history page tests: (1) upload 3 documents as User A and assert all 3 appear on `/documents`, (2) upload 1 document as User B and assert User A's history does NOT include User B's document (isolation check), (3) delete a document via the DELETE endpoint and assert: the MongoDB record is removed, the file is removed from `uploads/`, and it no longer appears on the history page, (4) trigger re-analyze on a processed document and assert `status` resets and the processing pipeline runs again, (5) call the delete endpoint on a `doc_id` belonging to a different user and assert a 403 Forbidden response."

Phase 4 — AI Summarization

Prompt 8 — Document Summarization with LLM

> "In LegalEase, integrate an LLM to generate document summaries at `/document/<doc_id>/summary`. Build two prompt templates: one for a short summary (3–5 sentences) and one for a structured breakdown with sections: Purpose, Parties, Obligations, Duration, and Termination. Accept a `?mode=short|detailed` query param. Parse and store under `summary.short` and `summary.detailed` in MongoDB."

🧪 Test — Prompt 8

> "Write summary tests: (1) call `/document/<doc_id>/summary?mode=short` and assert the response is a JSON object with a `summary` string of 3–5 sentences (count sentences using a period-split heuristic), (2) call with `?mode=detailed` and assert the response JSON contains all 5 keys: `Purpose`, `Parties`, `Obligations`, `Duration`, `Termination` and each is non-empty, (3) assert that after calling both modes, MongoDB stores both under `summary.short` and `summary.detailed`, (4) call the endpoint a second time and assert it returns the cached MongoDB result instead of making a redundant LLM API call (add a `cache_hit` flag in the response), (5) call the endpoint for a `doc_id` with `status` = 'uploaded' (not yet processed) and assert a 409 Conflict or a meaningful error response."

Phase 5 — Clause Extraction & Risk Analysis

Prompt 9 — Key Clause Extraction

> "In LegalEase, build a `/document/<doc_id>/clauses` route. Use the LLM to extract key clauses from `cleaned_text`. The LLM should return a structured JSON list where each item has: `clause_id`, `title`, `text`, `category`. Store the array under `clauses` in the MongoDB document. Render a `/document/<doc_id>/analysis` template that displays each clause as a card."

🧪 Test — Prompt 9

> "Write clause tests: (1) call the clauses endpoint on a processed NDA document and assert the response contains at least 3 clause objects, (2) assert every clause object has all required fields: `clause_id`, `title`, `text`, `category` — none should be null or empty, (3) assert `clauses` is written to the MongoDB document correctly, (4) hit the analysis page at `/document/<doc_id>/analysis` and assert it returns a 200 with clause card HTML elements in the response body, (5) call the endpoint for a non-existent `doc_id` and assert a 404, (6) simulate the LLM returning malformed JSON and assert the endpoint handles the parse error gracefully without a 500 crash."

Prompt 10 — Risk Flagging & Risk Summary Card

> "Extend clause extraction to add risk scoring. For each clause, have the LLM assign a `risk_level` (Low, Medium, High) with a short `risk_reason`. Display a Risk Summary card at the top of the analysis page that counts clauses by risk level and highlights all High-risk clauses. Store `risk_level` and `risk_reason` inside each clause object in MongoDB."

🧪 Test — Prompt 10

> "Write risk flagging tests: (1) assert every clause object in the response has a `risk_level` that is strictly one of 'Low', 'Medium', or 'High' — reject any other string, (2) assert every clause has a non-empty `risk_reason`, (3) on the analysis page, assert the Risk Summary card is rendered and shows correct counts (e.g. if 2 High clauses exist, the card shows '2 High'), (4) simulate a document with a clause containing 'indemnify against all claims' and assert it is flagged as High risk, (5) simulate the LLM returning a risk_level value outside the allowed set and assert it is normalized or rejected with a logged warning rather than stored as-is."

Prompt 11 — Manual Annotations, Risk Overrides & Export

> "Add an annotation system to the analysis page. A user with role 'legal_professional' can add text annotations to clauses and override the AI risk level. Store under `clauses[i].annotation` and `clauses[i].risk_override` in MongoDB. Add an export button that generates and downloads a PDF report using ReportLab or WeasyPrint."

🧪 Test — Prompt 11

> "Write annotation and export tests: (1) as a 'legal_professional' user, POST an annotation to a clause and assert `clauses[i].annotation` is updated in MongoDB, (2) override a clause's risk from 'Low' to 'High' and assert `risk_override` is stored while `risk_level` (the original AI value) remains unchanged, (3) as a regular 'user' (not legal_professional), attempt the same annotation POST and assert a 403 Forbidden response, (4) call the export endpoint and assert the response has `Content-Type: application/pdf` and the body is a non-zero-length binary, (5) open the generated PDF and assert it contains the document title and at least one clause heading (use PyPDF2 in the test to read the PDF text)."

Phase 6 — RAG Chat Interface

Prompt 12 — Embeddings & RAG Pipeline Setup

> "Set up a RAG pipeline for document Q&A. For each processed document, chunk `cleaned_text` into overlapping 300-token segments. Generate embeddings using OpenAI `text-embedding-3-small` and store in a `chunks` MongoDB collection with fields: `doc_id`, `chunk_index`, `text`, `embedding`. Create a retrieval function that embeds a query and performs cosine similarity search to return the top-3 relevant chunks."

🧪 Test — Prompt 12

> "Write RAG pipeline tests: (1) process a document and assert the `chunks` collection contains records with the correct `doc_id` and all required fields, (2) assert that chunks are overlapping — consecutive chunks should share at least 1 sentence (check by comparing last sentence of chunk N with first sentence of chunk N+1), (3) call the retrieval function with a query that is semantically present in the document and assert the top result's `text` contains relevant content, (4) call the retrieval function with a completely unrelated query (e.g. 'what is the weather in Paris') and assert the top result has a cosine similarity below 0.5 (captures the out-of-scope case), (5) assert that re-processing a document clears and recreates its chunks rather than duplicating them."

Prompt 13 — Chat Interface & Conversation History

> "Build an interactive chat panel on the analysis page. The `/document/<doc_id>/chat` POST endpoint should embed the user's question, retrieve top chunks, pass them to the LLM with a context-only system prompt, and return the answer and source snippet. Maintain conversation history in the Flask session. Return a graceful fallback message for out-of-scope questions."

🧪 Test — Prompt 13

> "Write chat tests: (1) POST a question directly about the document content and assert the response JSON has `answer` (non-empty) and `source_snippet` (non-empty), (2) assert the source snippet is a

> substring of the document's stored `cleaned_text`, (3) ask a follow-up question that references a term from the previous answer and assert it is answered correctly (verifying session history is used), (4) POST a question completely unrelated to the document (e.g. 'Who won the World Cup?') and assert the response contains the fallback message string, (5) send 10 rapid sequential messages and assert no session corruption or duplicate history entries occur, (6) call the endpoint as an unauthenticated user and assert a 401 or redirect."

Phase 7 — Document Comparison

Prompt 14 — Multi-Document Comparison

> "Build a comparison feature. On the `/documents` page, add checkboxes to select 2–4 documents and a 'Compare' button that POSTs selected `doc_ids` to `/compare`. The comparison page renders a side-by-side clause table matched by category, and the LLM generates a comparison summary highlighting obligation differences and risk discrepancies."

🧪 Test — Prompt 14

> "Write comparison tests: (1) POST 2 valid `doc_ids` belonging to the logged-in user and assert a 200 response with a comparison table in the HTML, (2) POST only 1 `doc_id` and assert a 400 validation error with a message like 'Select at least 2 documents', (3) POST 5 `doc_ids` and assert a 400 error for exceeding the maximum, (4) POST a `doc_id` that belongs to a different user and assert a 403 Forbidden, (5) assert the comparison summary JSON contains at least one of: 'obligations', 'risk', or 'differences' as keys, (6) compare two identical documents and assert the summary notes no meaningful differences."

Phase 8 — Notifications

Prompt 15 — In-App & Email Notifications

> "Create a `notifications` MongoDB collection. Trigger notifications when document processing completes or a High-risk clause is detected. Display an unread badge in the navbar and a `/notifications` page. Integrate Flask-Mail for email alerts on High-risk clause detections."

🧪 Test — Prompt 15

> "Write notification tests: (1) upload and process a document and assert a notification record is created in MongoDB with correct `user_id`, `type`, and `message`, (2) process a document with a mocked High-risk clause and assert both an in-app notification and an email are triggered (mock Flask-Mail's `send` method and assert it was called with the correct recipient), (3) mark a notification as read via the appropriate endpoint and assert `read` = True in MongoDB, (4) assert the navbar badge count decrements by 1 after marking one notification read, (5) assert that User B's notifications are not visible on User A's `/notifications` page, (6) assert that if SMTP is misconfigured, the email failure is logged silently and does NOT crash the request."

Phase 9 — Admin Panel

Prompt 16 — Admin Dashboard & User Management

> "Build an admin panel at `/admin/*` protected by a role guard for `role = 'admin'`. The `/admin/users` page lists all users with document counts. Add actions to promote to 'legal_professional', suspend accounts (`is_active = False`), and view a user's document list."

🧪 Test — Prompt 16

> "Write admin panel tests: (1) access `/admin/users` as a regular user and assert a 403 Forbidden, (2) access it as an admin and assert a 200 with a list of registered users, (3) promote a user to 'legal_professional' and assert their `role` field is updated in MongoDB, (4) suspend a user and immediately try to log in as that user — assert a login failure with an 'account suspended' flash message, (5) unsuspend the same user and assert login works again, (6) assert that the document count shown for a user on the admin page matches the actual MongoDB count for that user."

Prompt 17 — Platform Analytics & AI Performance Metrics

> "Build `/admin/analytics` using MongoDB aggregation pipelines to compute: total documents, breakdown by doc_type, average risk score per doc type, and weekly upload trends. Render with Chart.js. Build `/admin/ai-metrics` where admins can flag incorrect AI outputs, which stores a `flagged_outputs` document in MongoDB."

🧪 Test — Prompt 17

> "Write analytics tests: (1) insert a known set of documents with controlled `doc_type` and risk data into the test DB, call `/admin/analytics` as JSON, and assert the aggregation counts match exactly, (2) assert that average risk scores map correctly (Low=1, Medium=2, High=3) by seeding a document with 2 High and 1 Low clause and asserting `avg_risk` ≈ 2.33, (3) insert documents across 3 different weeks and assert the weekly trend endpoint returns 3 separate data points, (4) flag an AI output via the `/admin/ai-metrics` flag endpoint and assert a `flagged_outputs` document is created in MongoDB with `doc_id`, `clause_id`, `flagged_by`, and `timestamp`, (5) assert that accessing `/admin/analytics` with an empty database returns zeroed-out values rather than a 500 error."

Unified Testing Principles

A few rules apply across all 17 prompts. Always use a **separate test MongoDB database** (set via `MONGO_URI` in a `.env.test` file) and drop it after each test run to prevent state bleed between tests. Mock all external API calls (OpenAI, Flask-Mail SMTP) using `unittest.mock.patch` so tests run fast and offline. For every endpoint, always include at least one **unauthenticated request test** and one **wrong-user ownership test** to cover auth edge cases consistently.